AI-Powered RAG System – Architecture Description Document

1. Executive Summary

This document describes a production-oriented Retrieval Augmented Generation (RAG) system designed to enable intelligent querying over document repositories.

The system allows users to upload PDF documents and interact with them using natural language. Unlike traditional search systems, it combines semantic retrieval with Large Language Model (LLM) reasoning to produce accurate, context-grounded responses.

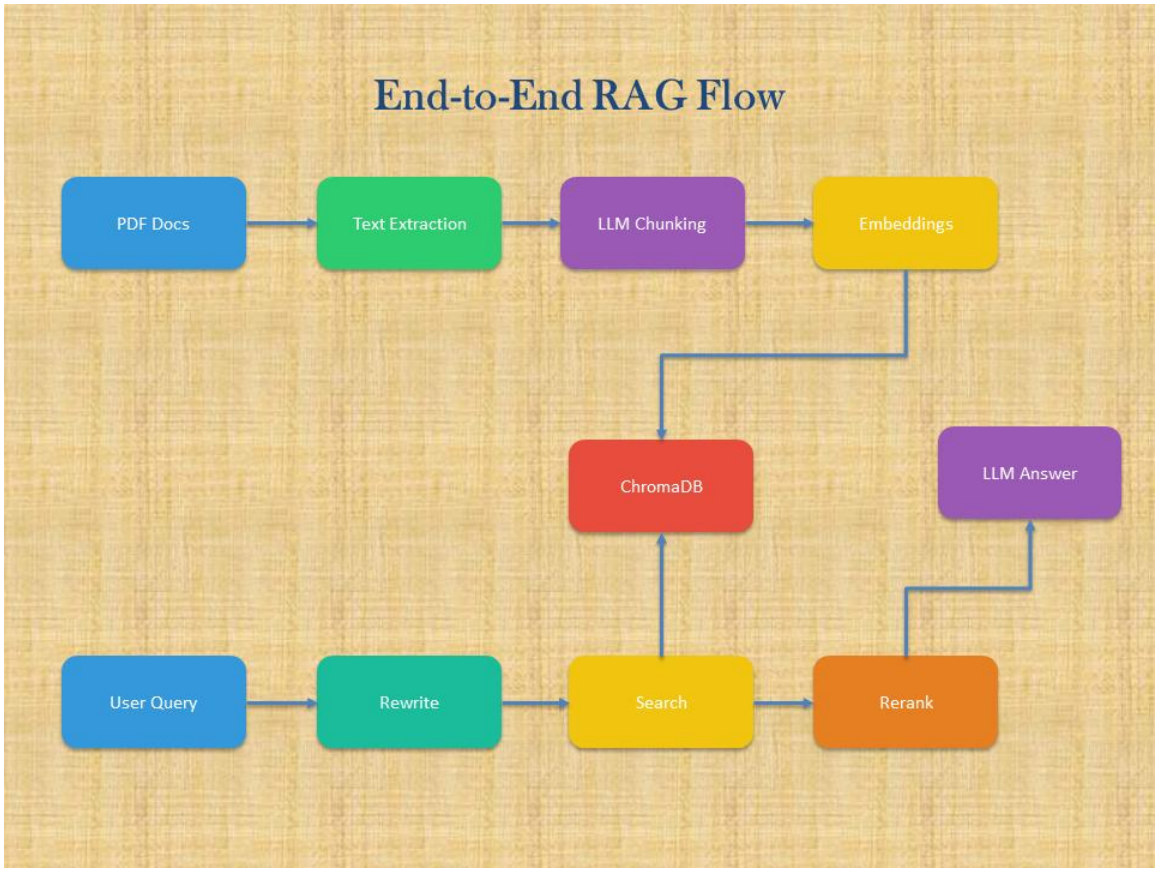
The solution demonstrates enterprise-grade design characteristics including modular architecture, configuration-driven behavior, observability through evaluation metrics, and extensibility for future enhancements.

The system is divided into two primary layers:
- Implementation Layer – Responsible for ingestion, retrieval, and answer generation
- Evaluation Layer – Responsible for measuring retrieval and answer quality

This separation aligns with modern AI system design principles where both capability and quality are first-class concerns.

2. System Architecture Overview

The system follows a standard Retrieval Augmented Generation (RAG) architecture.



Pipeline:
Document → Chunking → Embeddings → Vector Store → Query → Retrieval → Reranking → LLM → Response

Key architectural principles:
• Loose coupling between ingestion, retrieval, and UI layers
• Config-driven behavior using YAML
• Stateless query processing
• Resilient API calls using retry strategies
• Observability via evaluation metrics and dashboards

The architecture ensures scalability, maintainability, and adaptability for enterprise use cases.

3. End-to-End Processing Flow

1. Documents are ingested from a knowledge base or uploaded via UI.
2. Text is extracted from PDFs using PyMuPDF.
3. Documents are split into semantic chunks using an LLM.
4. Each chunk is embedded using an embedding model.
5. Embeddings and metadata are stored in ChromaDB.
6. User submits a query via UI.
7. Query is rewritten for improved retrieval.
8. Vector search retrieves top candidate chunks.
9. Retrieved chunks are reranked using an LLM.
10. Final context is constructed.
11. LLM generates an answer using the context.
12. Answer and sources are displayed in UI.
13. Evaluation layer measures retrieval and answer quality.

4. Implementation Layer

The implementation layer contains the runtime components responsible for delivering the RAG functionality.

Modules:

- UI.py – User interaction and visualization
- injest.py – Document ingestion pipeline
- answer.py – Retrieval and response engine
- property_loader_util.py – Configuration abstraction
- YAML files – Model and prompt configurations

This layer is designed to be modular, allowing independent evolution of ingestion, retrieval, and UI capabilities.

5. UI Layer – UI.py

The UI layer is implemented using Gradio and provides an interactive chatbot interface.

Responsibilities:

- Accept user queries
- Display streaming responses
- Show retrieved document sources
- Allow document uploads
- Display system metrics

Key design considerations:

- Streaming responses improve user experience

• Context transparency builds trust
• Integrated ingestion allows dynamic knowledge expansion

Key functions:
• chat_stream() – Streams LLM responses token by token
• process_uploaded_pdf_end_to_end() – Handles ingestion from UI
• format_context() – Formats retrieved chunks
• get_kb_stats() – Displays system metrics

The UI is designed for both usability and observability.

6. Ingestion Pipeline – injest.py

This module is responsible for building the knowledge base.

Responsibilities:
• Load PDF documents
• Extract text
• Generate semantic chunks using LLM
• Create embeddings
• Store vectors in ChromaDB

Key design decisions:
• LLM-based chunking improves semantic retrieval
• Overlapping chunks improve recall
• Multiprocessing improves ingestion performance
• Incremental ingestion supports dynamic updates

Core functions:
• fetch_documents() – Loads PDFs
• process_document() – Generates chunks
• create_chunks() – Parallel processing
• create_embeddings() – Builds vector store
• append_embeddings() – Incremental updates

This layer directly impacts retrieval quality and system performance.

7. Retrieval & Answer Engine – answer.py

This module implements the core intelligence of the system.

Responsibilities:
• Query rewriting
• Vector retrieval
• Context merging

• Reranking using LLM
• Answer generation

Key design features:
• Dual retrieval (original + rewritten query)
• LLM-based reranking for improved relevance
• Structured prompts for grounded responses
• Retry mechanisms for reliability

Core functions:
• rewrite_query() – Improves search queries
• fetch_context() – Retrieves relevant chunks
• rerank() – Orders chunks by relevance
• answer_question() – Generates final answer

This module is critical for ensuring high-quality responses.

8. Configuration Layer

Configuration is externalized using YAML files.

Files:
• model_details.yaml – Defines LLM and embedding models
• prompts.yaml – Defines system prompts
• example_questions_data.yaml – Sample queries

Utility:
• property_loader_util.py – Loads configuration values

Benefits:
• No hardcoding of models or prompts
• Easy environment-specific changes
• Improved maintainability and flexibility

9. Evaluation Layer

The evaluation layer provides systematic validation of system performance.

Components:
• eval.py – Core evaluation logic
• evaluator.py – Visualization dashboard
• test.py – Test definitions
• tests.jsonl – Test dataset

Purpose:
• Measure retrieval quality

• Evaluate answer accuracy
• Provide feedback for improvements

This layer ensures the system is not only functional but also measurable and optimizable.

10. Evaluation Metrics

Retrieval Metrics:
• Mean Reciprocal Rank (MRR)
• Normalized Discounted Cumulative Gain (nDCG)
• Keyword Coverage

Answer Metrics:
• Accuracy
• Completeness
• Relevance

Approach:
• Retrieval evaluated using statistical metrics
• Answers evaluated using LLM-as-a-judge

This hybrid evaluation approach combines quantitative and qualitative assessment.

11. Evaluation Dashboard – evaluator.py

Provides a visual dashboard using Gradio.

Features:
• Color-coded metrics (green/amber/red)
• Category-wise analysis
• Aggregated statistics
• Bar charts for performance visualization

This enables quick identification of system strengths and weaknesses.

12. Key Design Decisions

• Use of RAG instead of pure LLM – Ensures grounded answers
• LLM-based chunking – Improves semantic understanding
• Query rewriting – Enhances retrieval quality
• Reranking using LLM – Improves relevance
• YAML-driven configuration – Improves flexibility
• Separate evaluation layer – Enables continuous improvement

13. Scalability & Extensibility

The system can be extended in multiple ways:

- Replace ChromaDB with enterprise vector stores
- Integrate with enterprise document repositories
- Deploy as microservices
- Add authentication and access control
- Support additional document formats

The architecture supports horizontal scaling and modular enhancements.

14. Conclusion

This system represents a production-grade implementation of a RAG-based document assistant.

It demonstrates:

- Strong architectural separation
- High-quality retrieval and answer generation
- Integrated evaluation framework
- Config-driven design
- Extensibility for enterprise use

The design aligns with modern AI system architecture practices and is suitable for enterprise adoption in knowledge management, support systems, and intelligent search platforms.